

Pratibimb

Smart Xerox Printing Platform — Project Overview

A complete product & functional reference for the developer taking over the project. Explains what the platform does, who uses it, how every feature behaves, and what remains to be built — without requiring a read of the source code.

Product	Online document-printing marketplace
Live domain	www.pratibimb.online
Frontend	React (Vite) on Vercel
Backend	Node.js + Express on Render
Audience	Incoming developer / maintainer
Document type	Handover / product manual

Contents

1. Executive Summary
2. The Problem & The Solution
3. Target Users & Personas
4. User Roles & Permissions
5. System Architecture (Functional View)
6. Complete Feature Set
7. Functional Modules
8. Core Business Logic (Pricing, Settlement, Order Division, Load Balancing)
9. Order Lifecycle & Status Model
10. End-to-End User Journeys (Login → Logout)
11. Dashboards & Screens
12. The "Kit" Module (Study-Notes Ordering)
13. Automated Background Jobs
14. Real-Time Events & Notifications
15. Security & Data Privacy
16. Feature Dependencies
17. Real-World Use Cases
18. Project Scope & Current Implementation Status
19. Known Limitations
20. Pending Work & Future Enhancements
21. Glossary

1. Executive Summary

Pratibimb (internally "Smart Xerox Printing Platform") is an online marketplace that connects customers who need documents printed with nearby physical print/xerox shops. A customer uploads documents from a phone or laptop, configures exactly how each document should be printed (colour, sides, copies, page ranges, binding, lamination, and more), pays online, and later collects the finished prints from the chosen shop using a secure pickup code. On the shop side, orders arrive automatically, are routed to the correct physical printer, and print with minimal manual effort. The platform earns revenue by taking a configurable commission/margin on each order and settling the remainder to the shop.

The system is a production-grade, three-tier web application: a React single-page app (frontend), a Node.js/Express REST + Socket.IO API (backend), MongoDB for data, AWS S3 for document storage, Razorpay for payments, and direct **IPP** (Internet Printing Protocol) communication to shop printers. It is already deployed and operational. A secondary product line, the **Kit module**, lets college students order pre-compiled study-note "kits" for their course.

How to read this document. Sections 1–5 give the big picture. Sections 6–17 describe every feature and behaviour in product terms. Sections 18–20 cover status, limitations, and the roadmap — read these first if your job is to decide what to build next.

2. The Problem & The Solution

2.1 The problem

Getting documents printed at a local shop is slow and awkward. Customers must travel to the shop, wait in a queue, verbally explain their print requirements (which pages, how many copies, colour or black & white, single or double sided, binding, etc.), hand over a pen-drive or email, and wait while the operator figures it all out. Mistakes are common, sensitive documents are exposed on shared machines, and there is no price transparency up front. Shop owners, in turn, juggle walk-ins, phone orders, and manual payment collection with no structured queue or record-keeping.

2.2 The solution

Pratibimb moves the entire interaction online while keeping fulfilment local:

- **For customers:** discover nearby shops, upload documents, choose precise print options, see the exact price before paying, pay online, and collect with a QR/OTP pickup code — no waiting or explaining at the counter.
- **For shops:** receive a clean, priced, queued order; have it printed automatically on the right printer; track everything on a dashboard; and get paid through automated settlement.
- **For the platform:** aggregate demand, take a commission, and provide trust (verified shops, ratings, secure pickup, automatic refunds).

Privacy is a first-class concern: documents are stored temporarily in private cloud storage, accessed by the shop only through short-lived signed links, and automatically deleted after the order is complete.

3. Target Users & Personas

Persona	Who they are	What they want
Student	College/university students printing assignments, notes, projects, forms.	Cheap, fast, correct prints; binding for projects; study-note kits.
Working professional	Office workers printing contracts, tickets, ID photos, reports.	Convenience, colour accuracy, passport-photo grids, urgent turnaround.
Shopkeeper	Owner/operator of a local xerox/print shop.	More orders, less manual work, reliable printing, timely payouts.
Platform admin	Pratibimb operator/business owner.	Onboard & verify shops, set margins, monitor revenue, resolve issues.
Kit customer	First-year college students needing subject notes.	Order a ready-made bundle of notes for their college/department/year.

The platform is built for the **Indian market**: prices are in INR, phone numbers use the Indian 10-digit format, payments use Razorpay (UPI/cards/netbanking), and settlement supports UPI and bank transfer.

4. User Roles & Permissions

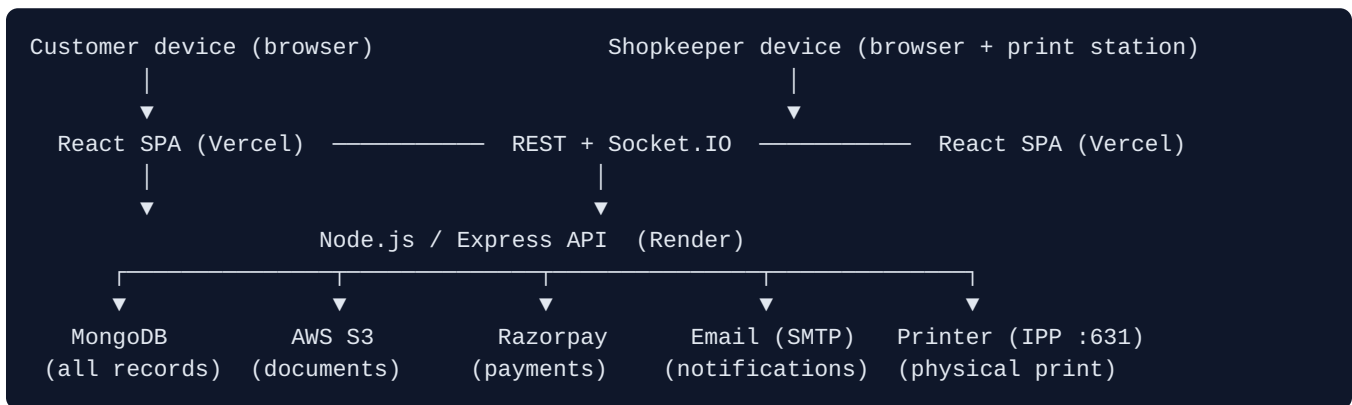
There are three primary roles stored on the user account (`user` , `shopkeeper` , `admin`). Access to every API and screen is gated by role.

Capability	Customer (user)	Shopkeeper	Admin
Register / login / manage own profile	✓	✓	✓
Browse shops, upload documents, place & pay for orders	✓	—	—
Track own orders, extend expiry, rate orders	✓	—	—
Create & manage a shop, set prices & hours	—	✓	—
Accept / reject / print orders, verify pickup	—	✓	—
Manage printers, view settlements, request withdrawals	—	✓	—
Verify shops, set platform margin, manage users	—	—	✓
View platform analytics/revenue, broadcast notices, process payouts	—	—	✓

A shopkeeper owns exactly one shop; ownership is enforced on every shop/printer/order action (a shopkeeper can only touch their own shop's data). Accounts can be deactivated by an admin, which blocks login and activity. Email and phone are independently verifiable, and sensitive actions require OTP.

5. System Architecture (Functional View)

This is a behavioural view — not a code walkthrough. Each box is a responsibility area a product manager would recognise.



- **Frontend** — the customer, shopkeeper, and admin experiences (one app, role-aware screens).
- **Backend API** — all business logic: auth, orders, pricing, payments, printing, settlement.
- **MongoDB** — users, shops, orders, payments, printers, reviews, notifications, settings.
- **AWS S3** — uploaded documents, accessed via short-lived signed URLs; auto-cleaned.
- **Razorpay** — payment collection, signature verification, webhooks, refunds.
- **IPP printing** — the backend sends the finished document straight to the shop's printer.
- **Socket.IO** — live order/printer/payment updates pushed to dashboards in real time.

Connectivity note (important for the next developer). The backend runs in the cloud but printers live on private shop networks. Reaching them requires a tunnel/VPN (Cloudflare Tunnel or Tailscale) or an on-site print agent. This is the single biggest operational consideration and is covered in Sections 19–20.

6. Complete Feature Set

The following is the full inventory of user-facing and system features, grouped by area. Each is described in product terms; detailed behaviour follows in later sections.

6.1 Accounts & Authentication

- Registration as customer or shopkeeper (name, email, Indian phone, password).
- Email verification via one-time password (OTP, 5-minute expiry).
- Login by email + password, or passwordless login by phone OTP.
- JWT access tokens with refresh tokens for long-lived sessions.
- Forgot/reset password via OTP; change password while logged in; change email (with verification).
- Account recovery flow and account lock/unlock protection against brute force.
- Admin can activate/deactivate any account.

6.2 Shop Discovery

- Browse all shops or search **nearby shops by geolocation** (latitude/longitude + radius).
- View shop details: address, services, prices, opening hours, rating, open/closed state.
- Read customer reviews and ratings for a shop.

6.3 Document Upload

- Upload one or up to five documents per order to private cloud storage.
- Supported types: PDF, DOC, DOCX, PPT, PPTX, and images (JPG/PNG).
- Automatic **page-count detection** for accurate pricing.
- Automatic file-type awareness: presentation files and image files unlock special print options.

6.4 Print Configuration (per document)

- **Page ranges** — print all or specific ranges (e.g. pages 3–10), with multiple ranges per document.
- **Copies** — 1 to 100 per range.
- **Colour mode** — black & white or colour (can differ per range within one order).
- **Sides** — single- or double-sided.
- **Pages per sheet (N-up)** — 1, 2, 4, 6, 9, or 16 pages per sheet.
- **Paper size** — A4, A3, or Letter; orientation portrait/landscape/auto.
- **Presentation options** — handout layouts, slides-per-page, print speaker notes, frame slides, scale to fit, include/exclude hidden slides, collation.
- **Image options** — full-page, **passport-photo grid**, **stamp grid**, or custom size (cm); plain or glossy paper; optional cut lines.

6.5 Additional Services

- Spiral binding, "blackbook" (project book), lamination, and urgent printing — each priced per the shop's rate card and added to the order total.
- Free-text special instructions (up to 500 characters).

6.6 Pricing & Payment

- Transparent, itemised price computed before payment from the shop's rate card.
- Online payment via Razorpay with server-side signature verification.
- Razorpay webhooks for reliable payment confirmation; automatic reconciliation.
- Refunds (manual and automatic on expiry) back to the original payment method.

6.7 Order Fulfilment & Printing

- Automatic dispatch to the shop when it is open; queued for auto-dispatch when the shop is closed.
- Shopkeeper can accept or reject orders and drive status manually if needed.
- **Direct IPP printing** to the assigned printer with correct colour/sides/paper/range mapping.
- **Load balancing** across multiple printers of the same type by current load and queue.
- **Order division** — mixed colour/B&W orders are split into sub-orders routed to the right printer type automatically.
- **Checkpointed printing with retry** — progress is tracked per document/range/copy so a failed job can resume and is retried with exponential backoff before being marked failed.
- **OTP stamping** — the pickup code is stamped onto the printout (configurable placement) so the shop can match prints to the collecting customer.

6.8 Secure Pickup

- On payment, the customer receives a **QR code and a numeric pickup code**.
- Shopkeeper verifies pickup by scanning the QR, entering the code, manual confirmation, or auto-confirmation — recorded with method and verifier.

6.9 Printer Management (shopkeeper)

- Register printers, add a printer manually by IP, update a printer's IP, rename display names.
- Enable/disable printers; live status via heartbeat; stale printers auto-marked offline.
- Load-balancer statistics: current load, queue depth, estimated wait, throughput.

6.10 Reviews, Ratings & Notifications

- Customers rate completed orders (1–5) and shops; shop average rating is maintained.
- In-app notifications, email notifications, and real-time socket alerts.
- Admin broadcast notifications to all users; push-notification token capture (FCM-ready).

6.11 Settlements & Withdrawals (shopkeeper)

- Shop accrues an available balance (order revenue minus platform commission).

- Shopkeeper requests withdrawals via UPI or bank transfer; admin processes them.

6.12 Admin & Platform Controls

- Platform dashboard, order analytics, revenue reports.
- User management, shop verification, per-shop commission/margin setting.
- Global platform settings including the printer **routing strategy** (strict type match / load balanced / cost optimised) and whether colour printers may serve B&W jobs.

7. Functional Modules

Module	Responsibility	Primary users
Auth & Account Recovery	Registration, login (password + OTP), tokens, password/email changes, recovery, account lock.	All
Users	Profile, order history, personal statistics.	Customer
Shops	Shop CRUD, geosearch, hours, services, rate card, open/close, dashboard stats.	Shopkeeper, Customer
Uploads	Document upload to S3, signed URLs, page detection, orphaned-file cleanup.	Customer
Orders	Order creation, lifecycle, accept/reject, status, extend, rate, pickup verify, division, dispatch.	Customer, Shopkeeper
Payments	Razorpay orders, verification, webhooks, refunds, reconciliation.	Customer, System
Printers	Registration, heartbeat, manual add, load balancing, status, IPP printing.	Shopkeeper, System
Notifications	In-app, email, socket, broadcast, push token.	All
Admin	Analytics, revenue, user/shop management, margins, platform settings, broadcast.	Admin
OTP	Generation/verification for email/phone/login/reset/pickup.	All, System
Kit	Study-notes catalogue, kit ordering, screenshot payment, fraud checks, shop verification.	Student, Shopkeeper

8. Core Business Logic

8.1 Pricing engine

Each shop maintains a rate card: per-page prices for black-and-white and colour, each split by single/double sided, plus per-document binding, per-page lamination, and per-sheet glossy paper charges. An order's price is computed server-side (never trusted from the client):

```
Document price = shop per-page rate (by colour + sides) × effective pages × copies
Effective pages = ceil(pages / 2) for double-sided, else pages
Subtotal       = sum of all document prices
Additional      = binding + lamination + glossy + urgent charges
Platform margin = commission % (set by admin per shop) applied to the order
Total          = subtotal + additional + platform margin
Shop receivable = total - platform commission (the platform keeps the margin)
```

The commission percentage in force is snapshotted onto the order at creation time, so later margin changes never retroactively alter historical orders.

8.2 Settlement

When an order completes, the shop's **available balance** increases by its receivable amount and the platform retains its commission. Shopkeepers request withdrawals (UPI or bank transfer); an admin marks each withdrawal pending → processing → completed (or rejected), recording a snapshot of the payout details and an optional transaction reference.

8.3 Order division (mixed colour handling)

A single order can contain both colour and black-and-white pages. Because a physical printer is either a colour or a B&W device, the system inspects each order and, when it contains both, **splits it into sub-orders** (a colour sub-order and a B&W sub-order) linked to a parent order. Each sub-order is then routed to a printer of the matching type. The parent order is only considered complete when all sub-orders finish.

8.4 Printer load balancing

When an order (or sub-order) is ready to print, the system selects the best printer of the required type using a weighted score that favours printers with lower current page-load and shorter job queues. Colour jobs are sent only to colour printers and B&W jobs only to B&W printers (a platform setting can relax this). When a new printer comes online, queued jobs on overloaded peers are rebalanced onto it.

8.5 Checkpointed printing & retry

Every print job tracks how far it has progressed (current document, range, copy, and pages printed). If a transient error occurs (timeout, connection refused, host unreachable), the job is retried with exponential backoff (2s, 4s, 8s). Only after retries are exhausted is the job marked failed, at which point the customer and shop are notified with an actionable message. A background job also re-attempts jobs left incomplete.

9. Order Lifecycle & Status Model

An order moves through a well-defined set of states. Understanding this state machine is essential for any new work on orders.

Status	Meaning
pending_payment	Order created; awaiting successful payment.
paid	Payment confirmed; pickup code & QR issued.
queued	Paid but shop is closed / no printer free; awaiting auto-dispatch.
accepted	Shop (or auto-dispatch) accepted; printer assigned.
printing	Job is being sent to / processed by the printer.
ready	Printed and awaiting customer pickup.
picked_up	Collected by the customer (pickup verified).
rejected	Shop declined the order (triggers refund).
expired	Not collected/printed within the allowed window (auto-refund).

cancelled / refunded	Cancelled or money returned.
----------------------	------------------------------

```

pending_payment →pay→ paid → (shop open?) →yes→ accepted → printing → ready → picked_up
                    ↳no→ queued →(shop opens)→ accepted ...
any active state →shop declines→ rejected (refund)
any active state →time window elapses→ expired (auto-refund)

```

Orders expire after a configurable window (default 12 hours) and can be extended by the customer (a further 12 hours). Expiry alerts are sent before expiry; on expiry an automatic refund is initiated.

10. End-to-End User Journeys (Login → Logout)

10.1 Customer journey

1. **Login/Register.** The customer signs up (verifying email via OTP) or logs in by password or phone OTP. A session is established with access + refresh tokens.
2. **Find a shop.** They browse or search nearby shops, compare prices/ratings/hours, and open a shop.
3. **Upload documents.** They add up to five files; the system stores them privately and detects page counts.
4. **Configure printing.** For each document they set page ranges, copies, colour, sides, N-up, paper size, and any presentation/image-specific options, plus add-ons (binding, lamination, urgent).
5. **Review price.** The itemised total (including platform margin) is shown before payment.
6. **Pay.** They pay through Razorpay; the server verifies the payment and issues a pickup QR + code.
7. **Automatic fulfilment.** If the shop is open the order is dispatched and printed automatically; if closed it waits in the queue and prints when the shop opens.
8. **Track & collect.** They watch live status updates, receive notifications, then collect the prints by presenting the QR/code. Optionally they extend an order nearing expiry.
9. **Rate.** After collection they rate the order/shop (1–5).
10. **Logout.** The session ends; refresh token is invalidated.

10.2 Shopkeeper journey

1. **Login** as a shopkeeper and land on the shop dashboard.
2. **Set up the shop** (first time): address & geolocation, opening hours, services, rate card, and register printers (by IP). Await admin verification to go live.
3. **Open the shop** (toggle) to start receiving auto-dispatched orders.
4. **Handle orders:** new paid orders arrive live; the shop accepts (or rejects) them. Accepted orders print automatically on the assigned printer; the shopkeeper can monitor and, if needed, drive status manually or retry a failed print.
5. **Hand over prints:** when the customer arrives, verify the pickup code/QR and mark the order picked up.
6. **Manage money:** watch the available balance grow and request withdrawals via UPI/bank.
7. **Logout.**

10.3 Admin journey

1. **Login** as admin to the platform dashboard.
2. **Onboard shops:** review and verify (or reject) new shops; set each shop's commission/margin.
3. **Operate:** monitor analytics and revenue, manage users (activate/deactivate), inspect all orders, configure global platform settings (printer routing strategy), and broadcast notices.
4. **Settle:** process shopkeeper withdrawal requests.
5. **Logout.**

11. Dashboards & Screens

11.1 Customer

[Index/Home](#) [Services](#) [Shop detail](#) [Order builder](#) [User Dashboard](#) [Orders](#) — discover shops, build and pay for orders, and track order history/status.

11.2 Shopkeeper (single dashboard, multiple tabs)

Tab	Purpose
Overview	Key stats: pending orders, revenue, ratings, activity at a glance.
Orders	Live queue of incoming/active orders; accept, reject, print, verify pickup.
Printers	Register/manage printers, IPs, enable/disable, live load-balancer stats.
Profile	Shop details, hours, services, rate card, open/close toggle.
Settlements	Available balance, withdrawal requests and history.
Network	Printer/agent connectivity view.

11.3 Admin

Single admin dashboard with platform overview, analytics, revenue, users, shops (verification & margin), all-orders view, and broadcast tools.

11.4 Kit

A dedicated kit-order screen for students and a separate kit shop dashboard for verifying and fulfilling kit orders.

12. The "Kit" Module (Study-Notes Ordering)

The Kit module is a distinct product line aimed at college students. Instead of uploading their own files, students order a ready-made bundle ("kit") of subject notes for their college, year, and department.

How it works

1. **Browse the catalogue.** The student selects year, college, college part, department, and subjects; the system serves the matching notes/kit definition from a curated catalogue.
2. **Place an order.** They provide contact details and choose their kit; a total is computed.
3. **Pay by screenshot.** Payment is by UPI, and the student uploads a payment screenshot (stored in S3). The order enters **Pending Verification**.
4. **Fraud screening.** An automated fraud check (advanced v2) evaluates the payment screenshot/order for signs of fraud and flags suspicious orders for the shopkeeper.
5. **Shop verification.** The shopkeeper reviews kit orders, sees suspicious-order and fraud-stat views, verifies payment, and moves the order through Payment Verified → Accepted, with email updates to the student at each step. Pickup uses an OTP.

Note: Unlike the main flow (fully automated Razorpay payments), kit payments are manual UPI screenshots verified by a human, which is why a dedicated fraud-detection layer exists. Migrating kits to automated Razorpay payments is a strong future improvement (Section 20).

13. Automated Background Jobs

A set of scheduled jobs keeps the platform consistent without human intervention.

Job	Frequency	What it does
Expiry alerts	every 30 min	Warns customers of orders expiring within ~1 hour.
Order expiry	every 15 min	Marks overdue orders expired and auto-initiates refunds.
Pending-payment expiry	every 30 min	Cancels orders that were never paid.
S3 cleanup	daily 2 AM	Deletes documents from old completed/expired orders (privacy).
Auto-hide old orders	hourly	Tidies finished orders out of active views.
Auto-retry incomplete jobs	every 15 min	Resumes/retries print jobs left incomplete.
Mark stale printers offline	every 5 min	Flags printers that stopped sending heartbeats.
Reassign offline-printer jobs	every 10 min	Moves queued jobs off printers that went offline.
Orphaned upload cleanup	every 30 min	Removes uploaded files that never became orders.
Re-emit stale accepted orders	every 5 min	Re-notifies the print station of stuck accepted orders.
Queue health check	every 5 min	Watches for queue backlogs/health issues.
Webhook retry	every 10 min	Retries failed payment webhook processing.

14. Real-Time Events & Notifications

Dashboards update live over Socket.IO. Shopkeepers join their shop's "room" to receive order and printer events; customers receive status updates for their own orders. Representative events include new order, order status update, order expiring/expired/extended, payment success/failure, printer status update, print started/completed/error, and admin broadcast. In parallel, users receive in-app notifications and emails for key milestones (payment, acceptance, ready, expiry, refunds). A push-notification token is captured on the user for future mobile push.

15. Security & Data Privacy

- JWT authentication with refresh tokens; role-based access control on every endpoint.
- OTP verification for email/phone/login/reset (short expiry, attempt limits, account lock).
- Razorpay webhook signature verification; server-side payment signature checks.
- Documents in private S3, reached only via short-lived signed URLs, and auto-deleted after completion.
- Rate limiting (global and stricter auth/OTP/upload/order limits), input validation, MongoDB-injection sanitisation, Helmet security headers, CORS allowlist, and CSRF protection.
- Secrets/keys via environment variables; sensitive fields (bank details, tokens) are non-selectable by default.

16. Feature Dependencies

Key ordering constraints a developer must respect when changing the system:

- **Upload → Order → Payment → Dispatch → Print → Pickup** is the backbone chain; each step depends on the previous succeeding.
- **Shop verification** (by admin) gates a shop's ability to receive live orders.
- **Printer registration + correct type** is a prerequisite for any printing; without an assigned, reachable printer of the right type, an order cannot be fulfilled.
- **Commission/margin setting** feeds the pricing engine; it is snapshotted per order.
- **Payment confirmation** is what issues the pickup code and triggers dispatch.
- **Printer reachability (tunnel/VPN/agent)** underpins the entire print step in production.

17. Real-World Use Cases

- **Assignment rush.** A student uploads a 40-page PDF, prints pages 5–38 double-sided B&W with spiral binding, pays, and collects within the hour using the pickup code.
- **Mixed report.** A professional submits a report where the cover and charts are colour and the body is B&W; the system splits it into colour and B&W sub-orders on two printers automatically.
- **Passport photos.** A customer uploads a photo, selects the passport-photo grid on glossy paper with cut lines, and gets a sheet of correctly sized photos.
- **After-hours order.** A customer pays at midnight; the order queues and prints automatically the moment the shop opens next morning.

- **Study kit.** A first-year student orders the department notes kit, pays by UPI screenshot, and the shopkeeper verifies and hands it over with an OTP.

18. Project Scope & Current Implementation Status

Overall status: A mature, production-deployed platform. The frontend (Vercel) and backend (Render) are live on www.pratibimb.online. Core flows — accounts, shop discovery, upload, ordering, pricing, Razorpay payments, order lifecycle, notifications, dashboards, settlements, admin controls, the kit module, and the full IPP printing pipeline — are implemented.

Area	Status	Notes
Accounts & auth	Complete	Password + phone-OTP login, recovery, logout.
Shop discovery & profiles	Complete	Geosearch, hours, rate cards, reviews.
Upload & configuration	Complete	Rich per-document options incl. presentation/image.
Pricing & payments	Complete	Razorpay + webhooks + refunds + reconciliation.
Order lifecycle & dispatch	Complete	Auto-dispatch, division, checkpoint retry.
IPP printing	Complete & recently hardened	Attribute grouping, page-range, status checks, timeout, retry — fixed and verified.
Printer management	Complete	Register/heartbeat/manual add/load balancing.
Settlements/withdrawals	Complete	UPI/bank withdrawal request + admin processing.
Admin & platform settings	Complete	Verification, margins, routing strategy.
Kit module	Complete (manual payments)	Screenshot payment + fraud checks + OTP pickup.
Printer connectivity (cloud → LAN)	In progress	Requires tunnel/VPN/agent — main open item.

19. Known Limitations

- **Cloud-to-printer reach.** The cloud backend cannot reach a shop's LAN printer directly. Production printing depends on a network bridge (Cloudflare Tunnel, Tailscale, or an on-site print agent). Each option has trade-offs (domain/DNS setup, exposure, or building an agent). This is the most important operational limitation.
- **Manual kit payments.** Kit orders rely on UPI screenshots verified by a human, which is error-prone and needs the fraud-detection layer as compensation.
- **Third-party dependency vulnerabilities.** A dependency audit flagged issues (notably in the email library and a UUID/cron dependency). These need version upgrades tested in staging.
- **Single-shop-per-printer assumptions.** Printer routing assumes correct per-shop printer setup and accurate colour/B&W typing; misconfiguration blocks fulfilment.
- **Scanning service not implemented.** A "scanning" service flag exists on shops but is not built.
- **No native mobile app yet.** Push-notification tokens are captured but mobile push is not wired.

20. Pending Work & Future Enhancements

Near-term (recommended next)

- **Finalise printer connectivity.** Choose and standardise one approach. The most robust long-term option is an **outbound print agent** installed at each shop that pulls jobs from the backend and prints on the local network — no inbound exposure, no domain/VPN friction, and it reuses the existing printer register/heartbeat endpoints. Cloudflare Tunnel or Tailscale are viable interim bridges.
- **Alerting.** Notify the shop/admin when a shop has zero online printers of a needed type so orders never queue silently.
- **Dependency upgrades.** Patch the flagged libraries and add `npm audit /verify` to the deploy pipeline.

Medium-term

- **Automate kit payments** via Razorpay to remove manual screenshot verification and fraud risk.
- **Mobile push notifications** using the already-captured FCM tokens.
- **Implement the scanning service** and any other advertised-but-unbuilt shop services.
- **Richer analytics** for shops (peak hours, popular options) and platform (cohorts, retention).

Longer-term opportunities

- Multi-shop price comparison and automatic best-shop routing.
- Subscription/wallet for frequent customers; loyalty and referral programs.
- Delivery option (courier the prints) in addition to pickup.
- Expanded document processing (auto-format, watermarking, cloud-import from Drive/email).

21. Glossary

Term	Meaning
IPP	Internet Printing Protocol — how the backend talks to a printer over the network (port 631).
Sub-order	A colour or B&W piece of a divided mixed-colour order, linked to a parent order.
Pickup code / QR	Secure token the customer presents to collect prints; also stamped on the printout.
Heartbeat	Periodic signal from a printer/print station proving it is online.
Load balancing	Choosing the least-busy suitable printer for a job.
Commission / margin	The platform's percentage cut of an order; the rest is the shop's receivable.
Kit	A pre-compiled bundle of study notes ordered by students.
Print agent	A small program installed at a shop that connects outbound and prints locally (recommended future connectivity model).

End of document · Pratibimb — Smart Xerox Printing Platform · Project Overview & Functional Specification for developer handover.